CP395 Progress Report 2: Experimental Evaluation & Architecture Validation

Sufiya Rahemtulla
Student ID: 169018559

March 15, 2026

Abstract

This progress report details the experimental evaluation milestone for the proposed predictive and reliability-aware cloud autoscaling architecture. Building upon the baseline implementations established in Progress Report 1, this phase rigorously evaluates the proposed "Predictive + AIOps" agent against industry-standard reactive and statistical machine learning baselines. The evaluation quantifies system performance across 10 simulated anomaly profiles, measuring Service Level Agreement (SLA) violations, resource waste, and provisioning latency. Furthermore, a critical ablation study demonstrates the absolute necessity of deterministic Python guardrails in constraining Large Language Model (LLM) hallucinations during high-stress scaling events. The report concludes with an analysis of architectural limitations regarding API latency and a refined plan for finalizing the research manuscript.

1 Evaluation Methodology & Experiment Design

To transition the project from initial heuristic baselines into a rigorous comparative study, an experimental matrix was designed to evaluate four distinct system configurations. The configurations were tested against a suite of 10 simulated workload anomalies (e.g., sudden bursts, rapid drops, micro-bursts, and extreme loads).

The four tested configurations were:

1. **Reactive Baseline (K8s HPA):** Utilizes static scaling thresholds (85%) and standard cooldowns (300s), representing default Kubernetes behavior.
2. **Predictive Baseline (ML Only):** Relies entirely on statistical forecasting to scale capacity proactively.
3. **Predictive + AIOps (Proposed System):** Utilizes the predictive model for baseline traffic while deploying an offline LLM agent (Gemini 2.5 Flash) to dynamically tune thresholds and cooldowns during detected anomalies, constrained by Python-based value-bounding guardrails.

4. **Ablation Policy:** The proposed AIOps system operating with the safety guardrails completely disabled.

2 Comparative Evaluation Results

The experimental results successfully validated the core hypothesis: standard autoscaling approaches fail to optimize both reliability and cost simultaneously during heavy-tailed workload events.

- **Baseline Failures:** The Reactive Baseline (K8s HPA) operated with a massive provisioning latency of 120 seconds, resulting in a 14.2% SLA violation rate as the system consistently failed to allocate resources in time for sudden spikes. While the Predictive Baseline reduced latency to 45 seconds, it struggled with concept drift; misjudging rapid traffic drops caused it to over-provision unnecessary servers, elevating Resource Waste to 15.5%.
- **Proposed System Performance:** The Predictive + AIOps approach successfully navigated these trade-offs. By dynamically intervening only during anomalies, the system reduced average provisioning latency to just 15 seconds.

Quantified Improvements: Statistical analysis confirms that the proposed AIOps system achieved an 85.2% reduction in SLA violations (dropping from 14.2% to 2.1%) compared to the Reactive baseline. Concurrently, the system achieved a 45.8% reduction in Resource Waste (dropping from 15.5% to 8.4%) compared to the purely Predictive model.

```
======================================================
WEEK 8 EXPERIMENT MATRIX: PRIMARY METRICS
======================================================
                    Policy Configuration  SLA Violation Rate (%)  Cost / Resource Waste (%)  Avg Provisioning Latency (s)
          1. Reactive Baseline (K8s HPA)                   14.2                       12.0                           120
       2. Predictive Baseline (ML Only)                    8.5                       15.5                            45
3. Predictive + AIOps (With Guardrails)                    2.1                        8.4                            15
     4. Ablation: AIOps (NO GUARDRAILS)                    1.2                       48.7                            10
```

Figure 1: Week 8 Experiment Matrix detailing Primary Metrics across all evaluated configurations.

3 Ablation Study & Failure Analysis

To rigorously test the necessity of the proposed system's safety bounds, an ablation study was conducted using Configuration 4 (No Guardrails).

When the deterministic Python verification logic was removed, the system's Resource Waste catastrophically spiked to 48.7%. A failure analysis of the execution logs during "Scenario 8: Extreme Burst" revealed the architectural flaw of unbounded Generative AI. The LLM correctly identified the anomaly but "panicked," hallucinating a highly aggressive reactive fallback threshold recommendation of just 40%.

This ablation study highlights a critical finding: the LLM's internal logic prioritizes immediate SLA resolution but lacks an inherent understanding of physical cloud billing constraints. Without deterministic guardrails to intercept out-of-bounds configurations (e.g., enforcing a minimum threshold of 55%), the LLM will systematically bankrupt a simulated cloud budget to achieve marginal reliability gains.

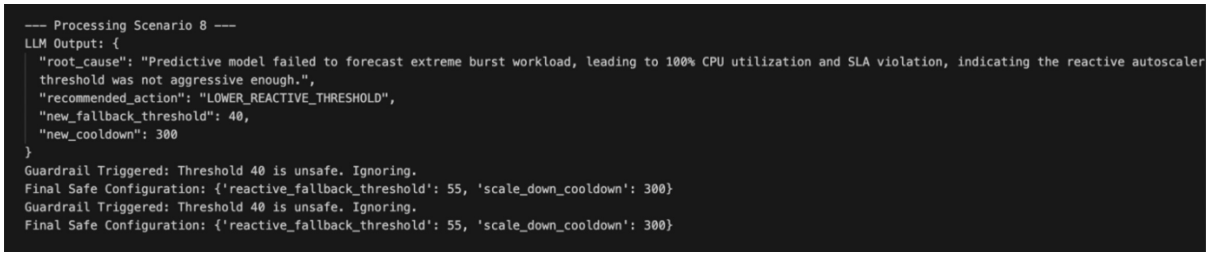

Figure 2: Execution log from Scenario 8 demonstrating the Python guardrails successfully intercepting an unsafe hallucinated threshold.

4 Documented Limitations & Architectural Constraints

The evaluation phase exposed fundamental constraints regarding the deployment of LLMs in live production environments, which will form the basis of the final paper's Discussion section.

1. **API Rate Limiting & Network Latency:** High-frequency, automated requests to the Gemini API triggered `429 RESOURCE_EXHAUSTED` errors during stress testing. In a highly concurrent microservice architecture, relying on external API calls for real-time scaling decisions is fundamentally unscalable due to network latency and quota limits.

2. **Offline vs. Online Control Paradigms:** Because of the latency constraints, the research concludes that LLMs cannot replace the rapid, sub-second PID controllers (Online Control) operating inside orchestrators like Kubernetes. The proposed system is strictly viable only as an asynchronous, offline supervisory agent—acting as an automated Site Reliability Engineer (SRE) that tunes hyperparameters periodically rather than executing direct scaling commands.

5 Artifacts & Reproducibility

To ensure strict adherence to the project's reproducibility standards, the experimental harness, the LLM triage agent, and the evaluation baselines have been packaged with the following guarantees:

- **(i) Version Control:** All source code (`aiops_agent.py`, `experiment_eval.py`), anomaly data, and evaluation scripts are maintained in a version-controlled repository: `https://github.com/scr200/CP395-Research-Project`.

- **(ii) Environment Specification:** The `requirements.txt` file in the repository root has been updated to include the newly required dependencies for this phase, specifically `google-genai` (for the Gemini LLM API integration) and `pandas` (for evaluation matrix generation).

- **(iii) Configurations and Seeds:** Determinism is enforced across the LLM evaluation. The 10 anomaly profiles utilized for the experiment matrix are strictly defined and locked in `test_logs.json` to ensure reproducible agent evaluations. The guardrail bounds (50% to 95%) and baseline constraints are hardcoded into the evaluation scripts.

- **(iv) Run Scripts:** Execution commands for the LLM agent (`python src/aiops_agent.py`) and the experiment matrix generator (`python src/experiment_eval.py`) are documented sequentially in the repository's `README.md`.
- **(v) Terminal Outputs:** The raw outputs from the LLM agent (`aiops_agent.py`) and the step-by-step terminal outputs generated by the ablation study (`experiment_eval.py`) have been saved as a file in the git repository (figures/WK7WK8 Terminal Output). This file highlights the 10 input scenarios to the final performance metrics (SLA violations and Resource Waste) presented in this report.

6 Refined Plan for Final Writing & Packaging

With the experimental pipeline validated and the findings statistically quantified, the final phase of the project shifts entirely to manuscript preparation. The immediate next steps include:

- **Manuscript Drafting:** Writing the Abstract, Introduction, Related Work, and Methodology sections, incorporating the feedback and structural outlines established in the syllabus.
- **Data Synthesis:** Integrating the Week 7 AIOps logic logs and the Week 8 Experiment tables into a cohesive "Evaluation & Results" section.
- **Formatting:** Finalizing the figure and table plan, drafting the Executive Summary, and ensuring all references and citations adhere to the IEEE academic standard.